Simulatore a eventi discreti di una coda M/M/c

Obiettivo

L'obiettivo di questo progetto è di realizzare un simulatore ad eventi di un sistema a coda della forma M/M/c, stimare le metriche L_q, L_x, L_s, W_q, W_s, P_{coda} e p_k e confrontarle con i rispettivi valori teorici.

Metodo

Per lo sviluppo del simulatore abbiamo scelto Rust per via delle sue elevate prestazioni e la sua sicurezza nella gestione della memoria.

L'architettura ruota attorno a tre struct principali:

- **Simulator**: che coordina l'intero processo di simulazione
- **Node**: che racchiude la logica relativa al buffer e ai server
- **Packet**: che raccoglie le informazioni di ogni pacchetto

Il calcolo delle statistiche è stato isolato nel modulo *collector.rs*, che ricava le metriche empiriche a partire dai dati della simulazione. Infine *utils.rs* contiene le funzioni per il calcolo dei valori teorici, da confrontare poi con quelli simulati.

Implementazione funzioni teoriche

Per quanto riguarda l'implementazione delle funzioni teoriche, abbiamo utilizzato formule diverse da quelle viste a lezione, poiché queste ultime risultavano poco stabili dal punto di vista algoritmico.

In particolare la formula della funzione B_{erlang}:

$$B(Y, A) = \frac{\frac{A^Y}{Y!}}{\sum_{i=0}^{Y} \frac{A^i}{i!}}$$

l'abbiamo riscritta come:

$$B(0, A) = 1$$

$$B(k, A) = \frac{A \cdot B(k-1, A)}{k + A \cdot B(k-1, A)}$$

Con la funzione B_{erlang} iterativa abbiamo ricavato la nuova formula della funzione C_{erlang}:

$$C(c, A) = \frac{B(c, A)}{1 - \frac{A}{c} \cdot (1 - B(c, A))}$$

Abbiamo calcolato la probabilità $P_k(t)$ in maniera iterativa:

$$v[0] = 1$$

$$\begin{cases} v[k] = v[k-1] \cdot \frac{A}{k} & k \leq c \\ v[k] = v[k-1] \cdot \rho & k > c \end{cases}$$

Successivamente si ha:

$$P_0 \cdot \sum_{k=0}^{\infty} v[k] = 1 \quad \Rightarrow \quad P_0 = \frac{1}{\sum_{k=0}^{\infty} v[k]}$$

$$\sum_{k=0}^{\infty} v[k] = \sum_{k=0}^{c-1} v[k] + \sum_{k=c}^{\infty} v[k]$$

con:

$$\sum_{k=c}^{\infty} v[k] = v[c] \cdot (1 + \rho + \rho^2 + \rho^3 + ...)$$

quella dentro alle parentesi è una serie geometrica che con $\rho < 1$ vale proprio $\frac{1}{1-\rho}$:

$$v[c] \cdot \frac{1}{1-\rho}$$

$$P_0 = \frac{1}{\sum_{k=0}^{c-1} v[k] + \frac{v[c]}{1-\rho}} \qquad p_k = P_0 \cdot v[k]$$

Implementazione funzioni empiriche

Durante la simulazione raccogliamo i dati necessari per il calcolo delle principali statistiche del sistema a coda M/M/c in forma empirica. Ci siamo, in particolare, concentrati sul calcolo dei principali indici caratteristici, quali:

- La probabilità di avere k pacchetti nel sistema (P_k)
- Il numero medio di pacchetti nel sistema (L_s)
- Il numero medio di pacchetti nella coda (L_q)
- Il numero medio di pacchetti in servizio (L_x)
- Il tempo medio che un pacchetto passa nel sistema (W_s)
- Il tempo medio che un pacchetto passa nella coda (W_q)
- La probabilità di un pacchetto di finire in coda (P_{coda})

P_k

Per il calcolo delle probabilità di avere k pacchetti nel sistema il metodo utilizzato consiste nel registrare, ad ogni gestione di un evento, il tempo **dt** che è passato dalla gestione dell'evento precedente (in modo da sapere per quanto tempo il sistema è rimasto in uno specifico stato), insieme al numero di pacchetti che erano presenti nella coda e nei server in quel lasso temporale **dt**.

In questo modo siamo in grado di salvarci, per un massimo di 48 istanze, per quanto tempo il sistema è rimasto con k pacchetti dall'inizio della simulazione. Dividendo ogni istanza per il tempo totale di simulazione abbiamo ottenuto la probabilità empirica di avere k pacchetti nel sistema.

L_s

Il metodo adoperato per il calcolo del numero medio di pacchetti nel sistema non si discosta eccessivamente dal precedente. L'idea alla base è quella di sommare il numero di pacchetti presenti nel sistema per ogni intervallo di tempo **dt** e dividere per il tempo totale di simulazione.

Questo ci permette di dare un "peso" alla quantità di pacchetti nel sistema basandoci su quanto tempo quei pacchetti rimangono nel sistema (ad esempio, 1 pacchetto presente per 10 secondi pesa quanto 10 pacchetti presenti per 1 secondo).

L_q

Il calcolo del numero medio di pacchetti in coda segue il medesimo ragionamento del calcolo di L_s, con l'unica differenza che ad essere sommato non è il numero di pacchetti nel sistema per ogni **dt**, ma è il numero di pacchetti nella coda per ogni **dt**.

La successiva divisione per il tempo totale di simulazione ci permette nuovamente di ottenere il valore di L_q desiderato.

L_x

Anche per il calcolo di L_x il metodo usato segue la stessa logica dei due precedenti, utilizzando questa volta il numero totale di pacchetti all'interno dei servitori per ogni **dt**.

W_s

Il metodo usato per il calcolo di W_s consiste nel sommare i tempi che i pacchetti passano nel sistema e dividere il tutto per il numero totale di pacchetti serviti.

Il tempo totale che un pacchetto passa nel sistema viene calcolato come la differenza tra il tempo della sua partenza ed il tempo del suo arrivo

W_q

Analogamente, per calcolare W_q, sommiamo i tempi che i pacchetti passano nella coda e dividiamo il totale per il numero di pacchetti serviti.

Il tempo totale che un pacchetto passa nella coda viene calcolato come la differenza tra il momento in cui ha iniziato ad essere servito, uscendo dalla coda, ed il tempo del suo arrivo.

Nel caso in cui il pacchetto in questione fosse stato servito immediatamente, il suo tempo di attesa risulterà nullo, contribuendo quindi a ridurre il tempo medio speso in coda.

P_{coda}

La probabilità di un pacchetto di finire in coda al suo arrivo è stata calcolata come il rapporto tra il numero totale di pacchetti accodati dall'inizio della simulazione ed il numero totale di arrivi.

Interarrival time distribution

Rappresentare l'andamento della distribuzione dei tempi di interarrivo ci ha permesso di verificare che questi seguono un andamento esponenziale.

Il metodo che abbiamo usato per registrare queste informazioni consiste nei seguenti passaggi:

- Definire un array di 48 posizioni che conterrà il numero di tempi di interarrivo che sono ricaduti in uno specifico range.
- Discretizzare i tempi ottenuti in intervalli di ampiezza **dipendente da λ**.

  $$\text{LAMBDA_BIN_WIDTH} = \frac{0.1}{\lambda}$$

  In questo modo possiamo ottenere un grafico sempre coerente con i tempi che ci aspettiamo di registrare. Senza questo passaggio, un λ troppo elevato diminuirebbe la media dei tempi di interarrivo a valori più piccoli di 0.1, riducendo il grafico della distribuzione dei tempi a una singola colonna $[0.0, 0.1)$.
- Inserire i tempi di interarrivo ottenuti all'interno delle posizioni corrette dell'array.
- Con gli elementi dell'array ottenere la densità di probabilità da confrontare con la curva teorica dei tempi di interarrivo.

Per ottenere la densità di probabilità abbiamo preso ogni elemento dell'array e l'abbiamo diviso per il numero totale di arrivi registrati nella simulazione moltiplicato per la larghezza dell'intervallo dell'istogramma.

Service time distribution

Per rappresentare l'andamento della distribuzione dei tempi di servizio abbiamo utilizzato la stessa tecnica usata per la distribuzione dei tempi di interarrivo, in quanto in un sistema M/M/c entrambi sono variabili aleatorie indipendenti e con distribuzione esponenziale (rispettivamente di parametro λ e μ).

Il metodo usato segue i medesimi passaggi precedenti:

- Definire un array di 48 posizioni che conterrà il numero di tempi di servizio che sono ricaduti in uno specifico range.
- Discretizzare i tempi ottenuti in intervalli di ampiezza **dipendente da μ**.

$$\text{MU_BIN_WIDTH} = \frac{0.1}{\mu}$$

- Inserire i tempi di servizio ottenuti all'interno delle posizioni corrette dell'array.
- Con gli elementi dell'array ottenere la densità di probabilità da confrontare con la curva teorica dei tempi di servizio.

Anche in questo caso, per ottenere la densità di probabilità abbiamo considerato ogni elemento dell'array e l'abbiamo diviso per il numero totale di partenze registrate nella simulazione moltiplicato per la larghezza dell'intervallo dell'istogramma.

Risultati

I risultati ottenuti da questo simulatore rispecchiano i valori teorici che ci aspettavamo di ottenere. Lo scorrere del tempo di simulazione e il continuo arrivo di nuovi eventi da processare permettono alla simulazione di raccogliere costantemente nuovi dati su cui calcolare le statistiche, e ci permettono di vedere graficamente come i valori tendano sempre di più a quelli teorici.

I grafici si assestano vicini alle curve teoriche, calcolate grazie alle formule, mentre i valori scalari riducono il loro errore rispetto al teorico man mano che la simulazione avanza.

Oltre al confronto diretto con i valori teorici, abbiamo osservato alcune proprietà che confermano la coerenza interna del simulatore. I valori ottenuti rispettano infatti la legge di Little ($L_s = \lambda W_s$, $L_q = \lambda W_q$), pur essendo L e W calcolati con metodi indipendenti: i primi come medie pesate sul tempo, i secondi come medie sui singoli pacchetti.

Abbiamo notato inoltre che non tutte le metriche convergono con la stessa velocità: L_x si stabilizza rapidamente, mentre L_q e W_q richiedono tempi di simulazione maggiori, poiché dipendono da periodi di congestione rari ma prolungati. Questo effetto si accentua all'avvicinarsi di ρ a 1.

Infine, per $k \geq c$ la distribuzione empirica p_k mostra l'andamento geometrico previsto dalla teoria ($p_k = p_c \cdot \rho^{k-c}$).

Grafici

Di seguito riportiamo i grafici ottenuti all'inizio e al termine della simulazione, confrontati con le rispettive curve teoriche.



